

Online Supermarket Database Project Report

Executive Summary

This report details the design and implementation considerations for an online ordering database system for a supermarket. The primary objective of this project is to develop a robust, scalable, and efficient database solution that facilitates online product browsing, order placement, inventory management, customer management, and delivery tracking. The system aims to enhance customer experience by providing a seamless online shopping platform, improve operational efficiency through streamlined inventory and order processing, and support data-driven decision-making for business growth.

The project encompasses a comprehensive approach, starting from a detailed logical design based on entity-relationship modeling, ensuring data integrity and normalization. This logical model is then translated into a physical design, outlining the concrete table structures, data types, primary and foreign key constraints, and indexing strategies optimized for performance. Key entities include Customers, Products, Orders, Order Items, Categories, Suppliers, Employees, and Deliveries. The report presents detailed descriptions of each table, highlighting their purpose, attributes, and relationships. Furthermore, a selection of critical SQL queries is provided, demonstrating how the database can be utilized to retrieve essential business information, such as customer order history, product availability, sales trends, and delivery statuses. These queries are accompanied by their expected result sets, illustrating the system's analytical capabilities.

By providing a solid foundation for managing complex supermarket operations, this database system is poised to significantly improve customer satisfaction, reduce manual errors, and empower the supermarket with the tools necessary to thrive in the competitive online retail landscape. The proposed design prioritizes data consistency, security, and performance, ensuring a reliable backbone for all online ordering activities.

1. Introduction

The retail industry has witnessed a profound transformation with the advent of e-commerce, and supermarkets are increasingly embracing online platforms to cater to evolving consumer preferences. An efficient and reliable database system is the cornerstone of any successful online ordering platform, managing a vast array of information from product catalogs and customer profiles to order details and inventory levels.

This project addresses the critical need for a well-structured database to support an online supermarket's operations. The current challenges in many existing systems include fragmented data, inefficient order processing, and a lack of real-time inventory updates, leading to customer dissatisfaction and operational bottlenecks.

1.1 Problem Statement

Traditional supermarket operations often struggle with managing diverse product inventories, tracking customer purchases, and coordinating deliveries efficiently when transitioning to an online model. Without a robust database, issues such as incorrect stock levels, delayed order fulfillment, and inaccurate customer data can severely impact business reputation and profitability. A comprehensive database solution is required to centralize data, automate processes, and provide insightful analytics.

1.2 Project Objectives

The primary objectives of this database project are:

- To design and implement a normalized database schema capable of storing all relevant information for an online supermarket.
- To ensure data integrity, consistency, and security through appropriate constraints and relationships.

- To support core functionalities such as customer registration, product catalog management, order placement, order tracking, payment processing, and delivery management.
- To provide efficient data retrieval mechanisms for reporting, analytics, and operational decision-making.
- To create a scalable foundation that can accommodate future growth and new features.

1.3 Scope of the Database

The database system will manage the following key areas:

- **Customer Management:** Storing customer profiles, addresses, contact information, and order history.
- **Product Management:** Maintaining product details, categories, pricing, stock levels, and supplier information.
- **Order Management:** Handling order creation, itemization, status tracking, and cancellation.
- **Inventory Management:** Updating stock levels in real-time based on sales and replenishments.
- **Payment Processing:** Recording payment details and transaction statuses (though not processing actual payments).
- **Delivery Management:** Assigning deliveries, tracking delivery status, and managing delivery personnel.
- **Employee Management:** Storing employee data for roles such as order pickers, packers, and delivery drivers.
- **Promotions/Discounts:** Managing various promotional offers.

2. Logical Design

The logical design phase focuses on defining the entities, their attributes, and the relationships between them, independent of any specific database management system (DBMS). This conceptual model ensures that all necessary data is captured and structured logically, adhering to principles of data integrity and minimizing redundancy.

2.1 Entity-Relationship Diagram (ERD) Description

The ERD for the online supermarket database identifies the following key entities and their relationships:

- **Customers:** Represents individual shoppers. Attributes: Address (PK), FirstName, LastName, Email, PhoneNumber, PasswordHash, RegistrationDate, DefaultShippingAddressID (FK).
- **Addresses:** Stores physical addresses for customers and potentially supermarkets/warehouses. Attributes: Address (PK), Stores (FK). A customer can have multiple addresses, and an address can belong to multiple customers (e.g., family). This implies a many-to-many relationship often resolved by a linking table or by making Addresses specific to Customers. For simplicity, we'll use (Addresses Address), (Addresses Address), (Addresses primary addresses and separate shipping addresses).
- **Products:** Represents items available for sale. Attributes: ProductID (PK), ProductName, Description, Price, StockQuantity, CategoryID (FK), SupplierID (FK), Weight, UnitOfMeasure, ImageURL, IsActive.
- **Categories:** Organizes products into logical groups. Attributes: CategoryID (PK), CategoryName, Description. A product belongs to one category.
- **Suppliers:** Information about product suppliers. Attributes: SupplierID (PK), SupplierName, ContactPerson, Email, PhoneNumber. A product is supplied by one supplier.
- **Orders:** Represents a customer's placed order. Attributes: OrderID (PK), CustomerID (FK), OrderDate, TotalAmount, OrderStatus (e.g., Pending, Processing, Shipped, Delivered, Cancelled), PaymentMethodID (FK), ShippingAddressID (FK), BillingAddressID (FK), DeliverySlotID (FK).

- **OrderItems:**Details the individual products within an order. Attributes includeOrderItemID(PK),OrderID(FK),ProductID(FK),Quantity,UnitPriceAtTimeOfOrder,LineTotal. This is a weak entity dependent onOrdersandProducts.
- **PaymentMethods:**Stores types of payment used (e.g., Credit Card, Debit Card, PayPal). Attributes includePaymentMethodID(PK),MethodName,Description. (Assumed generic for simplicity, not storing actual sensitive payment data).
- **Deliveries:**Manages the delivery process for orders. Attributes includeDeliveryID(PK),OrderID(FK),DeliveryDate,DeliveryStatus(e.g., Scheduled, Out for Delivery, Delivered),EmployeeID(FK - for delivery driver).
- **DeliverySlots:**Pre-defined time windows for delivery. Attributes includeDeliverySlotID(PK),StartTime,EndTime,Capacity,AvailableSlots.
- **Employees:**Information about supermarket staff, including delivery drivers, pickers, etc. Attributes includeEmployeeID(PK),FirstName,LastName,Role(e.g., Driver, Picker, Manager),ContactNumber.
- **Promotions:**Manages discount codes or offers. Attributes includePromotionID(PK),PromoCode,DiscountPercentage,StartDate,EndDate,MinOrderAmount,IsActive. An order might be linked to aPromotionID.

Relationships:

- ACustomerplaces manyOrders(1:M).
- AnOrdercontains manyOrderItems(1:M).
- AnOrderItemrefers to oneProduct(M:1).
- AProductbelongs to oneCategory(M:1).
- AProductis supplied by oneSupplier(M:1).
- AnOrderis associated with onePaymentMethod(M:1).
- AnOrderis associated with aShippingAddressandBillingAddress(M:1 toAddresses).
- AnOrderis fulfilled by oneDelivery(1:1, or 1:M if an order can have multiple delivery attempts). For simplicity, 1:1.
- ADeliveryis assigned to oneEmployee(driver) (M:1).
- AnOrderis scheduled for oneDeliverySlot(M:1).
- ACustomercan have multipleAddresses(1:M).

2.2 Normalization

The database design adheres to normalization principles, primarily up to the Third Normal Form (3NF), to eliminate data redundancy and improve data integrity.

- **First Normal Form (1NF):**All tables have a primary key, and all attributes are atomic (indivisible) with no repeating groups. For example,ProductNameis a single value, not a list of product names.
- **Second Normal Form (2NF):**Satisfies 1NF, and all non-key attributes are fully functionally dependent on the primary key. This means no partial dependencies exist. For instance, in theOrderItemstable,QuantityandUnitPriceAtTimeOfOrderdepend on the composite key (OrderID,ProductID), not justOrderIDorProductIDalone.
- **Third Normal Form (3NF):**Satisfies 2NF, and there are no transitive dependencies. Non-key attributes are not dependent on other non-key attributes. For example,CategoryNameis stored only in theCategoriestable, andProductIDin theProductstable links toCategoryIDrather than duplicatingCategoryNameinProducts. This prevents update anomalies where changing a category name would require updates across multiple product records.

Normalization helps in reducing storage space, preventing data anomalies (insertion, update, deletion), and making the database more flexible and easier to maintain.

2.3 Data Dictionary (Conceptual Overview)

A conceptual data dictionary provides a high-level overview of the main entities and their attributes:

Entity	Attributes (Example)	Description
Customers	CustomerID, FirstName, LastName, Email, PhoneNumber, RegistrationDate	Registered users of the online supermarket.
Products	ProductID, ProductName, Description, Price,	Items available for purchase.

	StockQuantity, CategoryID, SupplierID	
Categories	CategoryID, CategoryName, Description	Groups products (e.g., Dairy, Produce, Bakery).
Suppliers	SupplierID, SupplierName, ContactPerson, Email	Companies supplying products to the supermarket.
Orders	OrderID, CustomerID, OrderDate, TotalAmount, OrderStatus, ShippingAddressID	Placed orders by customers.
OrderItems	OrderItemID, OrderID, ProductID, Quantity, UnitPriceAtTimeOfOrder, LineTotal	Individual products within an order.
Addresses	AddressID, Street, City, State, PostalCode, Country	Physical addresses for customers, shipping, and billing.
Deliveries	DeliveryID, OrderID, DeliveryDate, DeliveryStatus, EmployeeID	Records the fulfillment and tracking of orders.
Employees	EmployeeID, FirstName, LastName, Role, ContactNumber	Staff involved in order fulfillment and delivery.
DeliverySlots	DeliverySlotID, StartTime, EndTime, Capacity	Pre-defined time slots for order delivery.
PaymentMethods	PaymentMethodID, MethodName	Generic payment types (e.g., Credit Card, UPI).

3. Physical Design

The physical design translates the logical model into a concrete schema that can be implemented in a specific DBMS (e.g., MySQL, PostgreSQL, SQL Server). This involves defining data types, primary and foreign keys, indexes, and other physical storage considerations.

3.1 Database Schema (DDL Concepts)

The physical design maps entities to tables, attributes to columns, and relationships to foreign key constraints.

- **Data Types:** Appropriate SQL data types are chosen for each attribute based on its nature (e.g., INT for IDs, VARCHAR for names, DECIMAL for prices, DATETIME for dates).
- **Primary Keys (PK):** Unique identifiers for each record in a table, often INT with auto-increment.
- **Foreign Keys (FK):** Enforce referential integrity, linking records between tables. ON DELETE and ON UPDATE actions (e.g., CASCADE, SET NULL, RESTRICT) are crucial for data consistency.
- **Constraints:**
 - NOT NULL: Ensures that a column cannot have a NULL value.
 - UNIQUE: Guarantees that all values in a column are distinct (e.g., Email in Customers).
 - CHECK: Defines conditions that must be met for data to be inserted or updated (e.g., StockQuantity > 0).
 - DEFAULT: Assigns a default value if no value is explicitly provided (e.g., OrderStatus defaults to 'Pending').
- **Indexes:** Created on primary keys automatically and recommended for foreign keys and frequently queried columns to speed up data retrieval.

3.2 Table Structures (DDL-like Descriptions)

Below is a textual representation of the table structures, outlining columns, data types, and key constraints.

```

-- Customers Table
CREATE TABLE Customers (
CustomerID INT PRIMARY KEY AUTO_INCREMENT,
FirstName VARCHAR(50) NOT NULL,
LastName VARCHAR(50) NOT NULL,
Email VARCHAR(100) UNIQUE NOT NULL,
PhoneNumber VARCHAR(20),
PasswordHash VARCHAR(255) NOT NULL,
RegistrationDate DATETIME DEFAULT CURRENT_TIMESTAMP,
DefaultShippingAddressID INT, -- FK to Addresses
DefaultBillingAddressID INT, -- FK to Addresses
INDEX idx_customer_email (Email)
);

-- Addresses Table
CREATE TABLE Addresses (
AddressID INT PRIMARY KEY AUTO_INCREMENT,
CustomerID INT NOT NULL, -- Link to customer who owns this address
Street VARCHAR(100) NOT NULL,
City VARCHAR(50) NOT NULL,
State VARCHAR(50),
PostalCode VARCHAR(10) NOT NULL,
Country VARCHAR(50) NOT NULL DEFAULT 'USA',
AddressType ENUM('Shipping', 'Billing', 'Home', 'Work') NOT NULL,
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE CASCADE
);

-- Categories Table
CREATE TABLE Categories (
CategoryID INT PRIMARY KEY AUTO_INCREMENT,
CategoryName VARCHAR(100) UNIQUE NOT NULL,
Description TEXT
);

-- Suppliers Table
CREATE TABLE Suppliers (
SupplierID INT PRIMARY KEY AUTO_INCREMENT,
SupplierName VARCHAR(100) UNIQUE NOT NULL,
ContactPerson VARCHAR(100),
Email VARCHAR(100),
PhoneNumber VARCHAR(20)
);

-- Products Table
CREATE TABLE Products (
ProductID INT PRIMARY KEY AUTO_INCREMENT,
ProductName VARCHAR(255) NOT NULL,
Description TEXT,
Price DECIMAL(10, 2) NOT NULL CHECK (Price >= 0),
StockQuantity INT NOT NULL CHECK (StockQuantity >= 0),
CategoryID INT NOT NULL,
SupplierID INT NOT NULL,
Weight DECIMAL(10, 2),
UnitOfMeasure VARCHAR(20),
ImageURL VARCHAR(255),
IsActive BOOLEAN DEFAULT TRUE,
FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID) ON DELETE RESTRICT,
FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID) ON DELETE RESTRICT,
INDEX idx_product_category (CategoryID),
INDEX idx_product_supplier (SupplierID),
INDEX idx_product_name (ProductName)
);

```

```

);
-- PaymentMethods Table (for recording types, not actual sensitive details)
CREATE TABLE PaymentMethods (
PaymentMethodID INT PRIMARY KEY AUTO_INCREMENT,
MethodName VARCHAR(50) UNIQUE NOT NULL,
Description TEXT
);
-- DeliverySlots Table
CREATE TABLE DeliverySlots (
DeliverySlotID INT PRIMARY KEY AUTO_INCREMENT,
StartTime DATETIME NOT NULL,
EndTime DATETIME NOT NULL,
Capacity INT NOT NULL CHECK (Capacity > 0),
AvailableSlots INT NOT NULL CHECK (AvailableSlots >= 0),
UNIQUE (StartTime, EndTime)
);
-- Employees Table (for delivery drivers, etc.)
CREATE TABLE Employees (
EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
FirstName VARCHAR(50) NOT NULL,
LastName VARCHAR(50) NOT NULL,
Role VARCHAR(50) NOT NULL, -- e.g., 'Driver', 'Picker', 'Manager'
ContactNumber VARCHAR(20) UNIQUE
);
-- Orders Table
CREATE TABLE Orders (
OrderID INT PRIMARY KEY AUTO_INCREMENT,
CustomerID INT NOT NULL,
OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
TotalAmount DECIMAL(10, 2) NOT NULL CHECK (TotalAmount >= 0),
OrderStatus ENUM('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled') DEFAULT 'Pending',
PaymentMethodID INT, -- Optional, if method chosen after order
ShippingAddressID INT NOT NULL,
BillingAddressID INT NOT NULL,
DeliverySlotID INT,
PromotionID INT, -- FK to Promotions table if promotions are used
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE RESTRICT,
FOREIGN KEY (PaymentMethodID) REFERENCES PaymentMethods(PaymentMethodID) ON DELETE SET NULL,
FOREIGN KEY (ShippingAddressID) REFERENCES Addresses(AddressID) ON DELETE RESTRICT,
FOREIGN KEY (BillingAddressID) REFERENCES Addresses(AddressID) ON DELETE RESTRICT,
FOREIGN KEY (DeliverySlotID) REFERENCES DeliverySlots(DeliverySlotID) ON DELETE SET NULL,
-- FOREIGN KEY (PromotionID) REFERENCES Promotions(PromotionID) ON DELETE SET NULL, (if
Promotions table exists)
INDEX idx_order_customer (CustomerID),
INDEX idx_order_status (OrderStatus),
INDEX idx_order_date (OrderDate)
);
-- OrderItems Table
CREATE TABLE OrderItems (
OrderItemID INT PRIMARY KEY AUTO_INCREMENT,
OrderID INT NOT NULL,
ProductID INT NOT NULL,
Quantity INT NOT NULL CHECK (Quantity > 0),
UnitPriceAtTimeOfOrder DECIMAL(10, 2) NOT NULL CHECK (UnitPriceAtTimeOfOrder >= 0),
LineTotal DECIMAL(10, 2) GENERATED ALWAYS AS (Quantity * UnitPriceAtTimeOfOrder) STORED, --
Computed column

```

```

FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE RESTRICT,
UNIQUE (OrderID, ProductID) -- Ensures one product per order item line
);
-- Deliveries Table
CREATE TABLE Deliveries (
DeliveryID INT PRIMARY KEY AUTO_INCREMENT,
OrderID INT UNIQUE NOT NULL, -- One delivery per order for simplicity
DeliveryDate DATETIME,
DeliveryStatus ENUM('Scheduled', 'Out for Delivery', 'Delivered', 'Failed', 'Cancelled') DEFAULT 'Scheduled',
EmployeeID INT, -- Delivery driver
FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID) ON DELETE SET NULL
);
-- Promotions Table (Optional, but useful for a full system)
CREATE TABLE Promotions (
PromotionID INT PRIMARY KEY AUTO_INCREMENT,
PromoCode VARCHAR(50) UNIQUE NOT NULL,
DiscountPercentage DECIMAL(5,2) NOT NULL CHECK (DiscountPercentage > 0 AND DiscountPercentage <= 100),
StartDate DATETIME NOT NULL,
EndDate DATETIME NOT NULL,
MinOrderAmount DECIMAL(10,2) DEFAULT 0.00,
IsActive BOOLEAN DEFAULT TRUE,
CHECK (EndDate > StartDate)
);
-- Link Promotions to Orders (add FK to Orders table once Promotions is defined)
-- ALTER TABLE Orders ADD FOREIGN KEY (PromotionID) REFERENCES Promotions(PromotionID) ON DELETE SET NULL;

```

3.3 Storage and Performance Considerations

- **Indexing:**Crucial for query performance. Primary keys are automatically indexed. Foreign keys and frequently used columns inWHEREclauses,JOINconditions, orORDER BYclauses should also be indexed (e.g.,EmailinCustomers,CategoryIDinProducts,OrderDateandOrderStatusinOrders). However, over-indexing can degrade write performance.
- **Data Types:**Choosing the most appropriate (and smallest possible) data types helps reduce storage space and improve query speed. For example, usingSMALLINTinstead ofINTif the range allows.
- **Normalization vs. Denormalization:**While 3NF is targeted, in some cases, for very high-read scenarios, controlled denormalization might be considered for performance, especially for summary tables or reporting needs. However, this is usually a later optimization.
- **Backup and Recovery:**Regular backups are essential. Strategies for full, differential, and transactional log backups should be planned.
- **Scalability:**The design uses auto-incrementing integer IDs, which are generally good for scalability. Future considerations might involve UUIDs for distributed systems or sharding strategies.

4. Table Descriptions

This section provides a detailed breakdown of each major table, including its purpose, columns, data types, constraints, and relationships.

1. Customers Table

- **Purpose:**Stores information about registered users of the online supermarket.

- **Columns:**
 - CustomerID:INT, Primary Key, Auto-increment. Unique identifier for each customer.
 - FirstName:VARCHAR(50), Not Null. Customer's first name.
 - LastName:VARCHAR(50), Not Null. Customer's last name.
 - Email:VARCHAR(100), Unique, Not Null. Customer's unique email address, used for login and communication.
 - PhoneNumber:VARCHAR(20), Optional. Customer's contact phone number.
 - PasswordHash:VARCHAR(255), Not Null. Hashed password for security.
 - RegistrationDate:DATETIME, DefaultCURRENT_TIMESTAMP. Date and time of customer registration.
 - DefaultShippingAddressID:INT, Foreign Key toAddresses.AddressID. Optional, default shipping address.
 - DefaultBillingAddressID:INT, Foreign Key toAddresses.AddressID. Optional, default billing address.
- **Relationships:**
 - One-to-Many withOrders(a customer can place many orders).
 - One-to-Many withAddresses(a customer can have multiple addresses).

2. Addresses Table

- **Purpose:**Stores physical address details for customers, used for shipping and billing.
- **Columns:**
 - AddressID:INT, Primary Key, Auto-increment. Unique identifier for each address.
 - CustomerID:INT, Not Null, Foreign Key toCustomers.CustomerID. Links an address to a specific customer.
 - Street:VARCHAR(100), Not Null. Street name and house number.
 - City:VARCHAR(50), Not Null. City of the address.
 - State:VARCHAR(50), Optional. State or province.
 - PostalCode:VARCHAR(10), Not Null. Postal or ZIP code.
 - Country:VARCHAR(50), Not Null, Default 'USA'. Country of the address.
 - AddressType:ENUM('Shipping', 'Billing', 'Home', 'Work'), Not Null. Specifies the type of address.
- **Relationships:**
 - Many-to-One withCustomers(many addresses can belong to one customer).
 - One-to-Many withOrders(as shipping/billing addresses).

3. Categories Table

- **Purpose:**Organizes products into logical classifications (e.g., 'Dairy & Eggs', 'Fresh Produce').
- **Columns:**
 - CategoryID:INT, Primary Key, Auto-increment. Unique identifier for each category.
 - CategoryName:VARCHAR(100), Unique, Not Null. Name of the category.
 - Description:TEXT, Optional. A brief description of the category.
- **Relationships:**
 - One-to-Many withProducts(a category can have many products).

4. Suppliers Table

- **Purpose:**Stores information about companies that supply products to the supermarket.
- **Columns:**
 - SupplierID:INT, Primary Key, Auto-increment. Unique identifier for each supplier.
 - SupplierName:VARCHAR(100), Unique, Not Null. Name of the supplier.
 - ContactPerson:VARCHAR(100), Optional. Main contact person at the supplier.
 - Email:VARCHAR(100), Optional. Supplier's email address.
 - PhoneNumber:VARCHAR(20), Optional. Supplier's phone number.
- **Relationships:**
 - One-to-Many withProducts(a supplier can supply many products).

5. Products Table

- **Purpose:**Contains details of all items available for sale in the supermarket.
- **Columns:**
 - ProductID:INT, Primary Key, Auto-increment. Unique identifier for each product.
 - ProductName:VARCHAR(255), Not Null. Name of the product.

- Description:TEXT, Optional. Detailed description of the product.
- Price:DECIMAL(10, 2), Not Null, Check (Price >= 0). Current selling price of the product.
- StockQuantity:INT, Not Null, Check (StockQuantity >= 0). Current quantity of the product in stock.
- CategoryID:INT, Not Null, Foreign Key toCategories.CategoryID. The category the product belongs to.
- SupplierID:INT, Not Null, Foreign Key toSuppliers.SupplierID. The supplier of the product.
- Weight:DECIMAL(10, 2), Optional. Weight of the product (e.g., in kg or lbs).
- UnitOfMeasure:VARCHAR(20), Optional. Unit for measurement (e.g., 'kg', 'piece', 'liter').
- ImageURL:VARCHAR(255), Optional. URL to the product image.
- IsActive:BOOLEAN, Default TRUE. Indicates if the product is currently available for sale.
- **Relationships:**
 - Many-to-One withCategories(many products in one category).
 - Many-to-One withSuppliers(many products from one supplier).
 - One-to-Many withOrderItems(a product can be part of many order items).

6. Orders Table

- **Purpose:**Records customer orders, their status, and total amount.
- **Columns:**
 - OrderID:INT, Primary Key, Auto-increment. Unique identifier for each order.
 - CustomerID:INT, Not Null, Foreign Key toCustomers.CustomerID. The customer who placed the order.
 - OrderDate:DATETIME, DefaultCURRENT_TIMESTAMP. Date and time the order was placed.
 - TotalAmount:DECIMAL(10, 2), Not Null, Check (TotalAmount >= 0). Total monetary value of the order.
 - OrderStatus:ENUM('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'), Default 'Pending'. Current status of the order.
 - PaymentMethodID:INT, Foreign Key toPaymentMethods.PaymentMethodID. The payment method used.
 - ShippingAddressID:INT, Not Null, Foreign Key toAddresses.AddressID. The address for delivery.
 - BillingAddressID:INT, Not Null, Foreign Key toAddresses.AddressID. The address for billing.
 - DeliverySlotID:INT, Foreign Key toDeliverySlots.DeliverySlotID. The chosen delivery time slot.
 - PromotionID:INT, Foreign Key toPromotions.PromotionID. Optional, applied promotion.
- **Relationships:**
 - Many-to-One withCustomers(many orders from one customer).
 - One-to-Many withOrderItems(an order contains many order items).
 - Many-to-One withAddresses(for shipping and billing).
 - Many-to-One withPaymentMethods(many orders use one payment method type).
 - Many-to-One withDeliverySlots(many orders can target one slot).
 - One-to-One withDeliveries(an order has one delivery record).
 - Many-to-One withPromotions(many orders can use one promotion).

7. OrderItems Table

- **Purpose:**Details the specific products and quantities within each order.
- **Columns:**
 - OrderItemID:INT, Primary Key, Auto-increment. Unique identifier for each item line in an order.
 - OrderID:INT, Not Null, Foreign Key toOrders.OrderID. The order this item belongs to.

- ProductID:INT, Not Null, Foreign Key toProducts.ProductID. The product included in this order item.
- Quantity:INT, Not Null, Check (Quantity > 0). Number of units of the product ordered.
- UnitPriceAtTimeOfOrder:DECIMAL(10, 2), Not Null, Check (UnitPriceAtTimeOfOrder >= 0). Price of the product when the order was placed (important for historical data if prices change).
- LineTotal:DECIMAL(10, 2), Generated Always As (Quantity * UnitPriceAtTimeOfOrder) Stored. Total price for this specific order item line.
- **Relationships:**
 - Many-to-One withOrders(many order items belong to one order).
 - Many-to-One withProducts(many order items refer to one product).

8. Deliveries Table

- **Purpose:**Tracks the delivery status and details for each order.
- **Columns:**
 - DeliveryID:INT, Primary Key, Auto-increment. Unique identifier for each delivery record.
 - OrderID:INT, Unique, Not Null, Foreign Key toOrders.OrderID. The order associated with this delivery.
 - DeliveryDate:DATETIME, Optional. Actual or planned delivery date and time.
 - DeliveryStatus:ENUM('Scheduled', 'Out for Delivery', 'Delivered', 'Failed', 'Cancelled'), Default 'Scheduled'. Current status of the delivery.
 - EmployeeID:INT, Foreign Key toEmployees.EmployeeID. The employee (driver) assigned to this delivery.
- **Relationships:**
 - One-to-One withOrders(one delivery for one order).
 - Many-to-One withEmployees(many deliveries assigned to one employee/driver).

9. Employees Table

- **Purpose:**Stores information about supermarket staff, particularly those involved in order fulfillment and delivery.
- **Columns:**
 - EmployeeID:INT, Primary Key, Auto-increment. Unique identifier for each employee.
 - FirstName:VARCHAR(50), Not Null. Employee's first name.
 - LastName:VARCHAR(50), Not Null. Employee's last name.
 - Role:VARCHAR(50), Not Null. Employee's role (e.g., 'Driver', 'Picker', 'Manager').
 - ContactNumber:VARCHAR(20), Unique. Employee's contact phone number.
- **Relationships:**
 - One-to-Many withDeliveries(an employee can be assigned many deliveries).

10. DeliverySlots Table

- **Purpose:**Defines available time windows for order delivery.
- **Columns:**
 - DeliverySlotID:INT, Primary Key, Auto-increment. Unique identifier for each delivery slot.
 - StartTime:DATETIME, Not Null. Start time of the delivery window.
 - EndTime:DATETIME, Not Null. End time of the delivery window.
 - Capacity:INT, Not Null, Check (Capacity > 0). Maximum number of orders that can be scheduled for this slot.
 - AvailableSlots:INT, Not Null, Check (AvailableSlots >= 0). Number of remaining available slots.
- **Relationships:**
 - One-to-Many withOrders(many orders can be scheduled for one delivery slot).

11. PaymentMethods Table

- **Purpose:**Lists the types of payment methods accepted by the supermarket.
- **Columns:**
 - PaymentMethodID:INT, Primary Key, Auto-increment. Unique identifier for each payment method type.

- MethodName:VARCHAR(50), Unique, Not Null. Name of the payment method (e.g., 'Credit Card', 'PayPal', 'Cash on Delivery').
- Description:TEXT, Optional. A brief description of the payment method.
- **Relationships:**
 - One-to-Many withOrders(many orders can use a specific payment method type).

12. Promotions Table (Optional)

- **Purpose:**Manages discount codes and promotional offers.
- **Columns:**
 - PromotionID:INT, Primary Key, Auto-increment. Unique identifier for each promotion.
 - PromoCode:VARCHAR(50), Unique, Not Null. The actual code customers enter.
 - DiscountPercentage:DECIMAL(5,2), Not Null, Check (DiscountPercentage > 0 AND DiscountPercentage <= 100). The percentage discount applied.
 - StartDate:DATETIME, Not Null. When the promotion becomes active.
 - EndDate:DATETIME, Not Null, Check (EndDate > StartDate). When the promotion expires.
 - MinOrderAmount:DECIMAL(10,2), Default 0.00. Minimum order total required to apply the promotion.
 - IsActive:BOOLEAN, Default TRUE. Indicates if the promotion is currently active.
- **Relationships:**
 - One-to-Many withOrders(one promotion can be applied to many orders).

5. Queries and Their Results

This section provides example SQL queries to demonstrate the functionality and data retrieval capabilities of the online supermarket database. Each query includes its purpose and a sample result set.

Sample Data for Demonstration:

To make the query results meaningful, let's assume some basic data:

Customers:

CustomerID	FirstName	LastName	Email	RegistrationDate
1	Alice	Smith	alice@example.com	2023-01-15 10:00:00
2	Bob	Johnson	bob@example.com	2023-02-20 11:30:00

Categories:

CategoryID	CategoryName
1	Dairy & Eggs
2	Fresh Produce
3	Bakery

Suppliers:

SupplierID	SupplierName
1	DairyCo
2	GreenFarm

Products:

ProductID	ProductName	Price	StockQuantity	CategoryID	SupplierID
101	Milk (1L)	2.50	100	1	1
102	Eggs (dozen)	3.00	50	1	1
103	Apples (1kg)	4.00	200	2	2
104	Bread (loaf)	3.50	80	3	1

Orders:

OrderID	CustomerID	OrderDate	TotalAmount	OrderStatus	ShippingAddressID	BillingAddressID	DeliverySlotID
---------	------------	-----------	-------------	-------------	-------------------	------------------	----------------

JOIN Suppliers S ON P.SupplierID = S.SupplierID
WHERE P.StockQuantity < 60;

- **Sample Result:** | ProductName | StockQuantity | SupplierName |
| :----- | :----- | :----- |
| Eggs (dozen) | 50 | DairyCo |
| Bread (loaf) | 80 | DairyCo | -- Error in sample data, Bread should be <60, correcting. Let's assume bread is 40.
Correction for sample data for Bread: Let StockQuantity for Bread (loaf) be 40. | ProductName
| StockQuantity | SupplierName | |
|---|---|---|
| Eggs (dozen) | 50 | DairyCo |
| Bread (loaf) | 40 | DairyCo |

Query 5: Calculate the total sales amount per customer.

- **Purpose:** To understand customer spending patterns and identify top customers.
- **SQL Query:** SELECT C.FirstName, C.LastName, SUM(O.TotalAmount) AS TotalSpent
FROM Customers C
JOIN Orders O ON C.CustomerID = O.CustomerID
GROUP BY C.CustomerID, C.FirstName, C.LastName
ORDER BY TotalSpent DESC;
- **Sample Result:** | FirstName | LastName | TotalSpent |
| :----- | :----- | :----- |
| Alice | Smith | 18.50 |
| Bob | Johnson | 7.00 |

Query 6: Get current delivery status for all 'Processing' orders, including driver details.

- **Purpose:** For logistics and customer service to track active deliveries.
- **SQL Query:** SELECT O.OrderID, C.FirstName AS CustomerFirstName, C.LastName AS CustomerLastName,
O.OrderStatus, D.DeliveryStatus, E.FirstName AS DriverFirstName, E.LastName AS DriverLastName, DS.StartTime, DS.EndTime
FROM Orders O
JOIN Customers C ON O.CustomerID = C.CustomerID
LEFT JOIN Deliveries D ON O.OrderID = D.OrderID
LEFT JOIN Employees E ON D.EmployeeID = E.EmployeeID
LEFT JOIN DeliverySlots DS ON O.DeliverySlotID = DS.DeliverySlotID
WHERE O.OrderStatus = 'Processing';
- **Sample Result:** | OrderID | CustomerFirstName | CustomerLastName | OrderStatus |
DeliveryStatus | DriverFirstName | DriverLastName | StartTime | EndTime |
| :----- | :----- | :----- | :----- | :----- | :----- | :----- | :----- | :----- |
| 2 | Bob | Johnson | Processing | Scheduled | (NULL) | (NULL) | 2023-03-05 16:00:00 |
2023-03-05 18:00:00 |
(Assuming Bob's order is 'Processing' but not yet assigned a driver in sample data for Delivery.)

Query 7: Update product stock after an order is placed (or delivered).

- **Purpose:** To decrement the stock quantity of a product after it's purchased.
- **SQL Query (Example: After Order 3 items are picked):** -- This would typically be part of a transaction.
-- For ProductID 101 (Milk): 1 item bought in Order 3
UPDATE Products
SET StockQuantity = StockQuantity - 1
WHERE ProductID = 101;
-- For ProductID 104 (Bread): 2 items bought in Order 3
UPDATE Products
SET StockQuantity = StockQuantity - 2
WHERE ProductID = 104;
- **Expected Result (conceptual):**

- ProductstableStockQuantityforProductID = 101would change from100to99.
- ProductstableStockQuantityforProductID = 104would change from40to38.

Query 8: List the most popular products by total quantity sold.

- **Purpose:**To identify best-selling products for marketing and inventory planning.
- **SQL Query:**SELECT P.ProductName, SUM(OI.Quantity) AS TotalQuantitySold
FROM OrderItems OI
JOIN Products P ON OI.ProductID = P.ProductID
GROUP BY P.ProductName
ORDER BY TotalQuantitySold DESC
LIMIT 5; -- Top 5 products
- **Sample Result:** | ProductName | TotalQuantitySold |
| :----- | :----- |
| Milk (1L) | 3 |
| Bread (loaf) | 3 |
| Eggs (dozen) | 1 |
| Apples (1kg) | 1 |

6. Conclusion and Future Work

6.1 Conclusion

This report has outlined a comprehensive database design for an online supermarket ordering system. The logical design, based on normalized entity-relationship modeling, ensures data integrity, minimizes redundancy, and provides a clear structure for storing all essential business information. The physical design details the implementation specifics, including table schemas, data types, and constraint definitions, optimized for a relational database management system. Through detailed table descriptions and illustrative SQL queries, the report demonstrates the system's capability to manage customer data, product catalogs, order processing, inventory, and delivery logistics efficiently. This robust database serves as a fundamental backbone, enabling the supermarket to offer a seamless online shopping experience, enhance operational efficiency, and gain valuable insights from its sales data.

6.2 Future Work

While the current design provides a solid foundation, several areas could be explored for future enhancements:

- **User Reviews and Ratings:**Integrate tables for customers to leave product reviews and ratings, enhancing social proof and product discoverability.
- **Wishlists and Favorites:**Allow customers to save products to wishlists for later purchase.
- **Recommendation Engine:**Implement features to suggest products based on past purchases or browsing history.
- **Real-time Inventory Synchronization:**Enhance inventory management with more sophisticated triggers or messaging queues for real-time stock updates across multiple sales channels (online, in-store).
- **Advanced Analytics and Reporting:**Develop dedicated data warehousing or business intelligence layers for complex analytical queries and dashboard reporting, beyond basic operational queries.
- **Payment Gateway Integration:**While the current design records paymentmethod, actual integration with secure payment gateways for processing transactions would be a critical next step for the application layer.
- **Promotions and Loyalty Programs:**Expand thePromotionstable to support more complex discount rules, coupon management, and loyalty point systems.
- **Multi-warehouse/Store Support:**Extend theProductsandInventorytables to manage stock across multiple physical locations.
- **Employee Permissions and Roles:**Implement a more granular employee management system with roles-based access control (RBAC) for different supermarket staff functions.

- **Audit Trails:**Add logging mechanisms to track changes to critical data for compliance and troubleshooting.
- **Caching Strategy:**For high-traffic areas like product browsing, implement a caching layer to improve response times and reduce database load.